

Lecture 34 — Embedded RTOS: FreeRTOS

Jeff Zarnett
jzarnett@uwaterloo.ca

Department of Electrical and Computer Engineering
University of Waterloo

September 7, 2026

We know a typical OS is not suitable for real-time systems.

It is also common for the real-time systems running the most life-and-safety critical things to be embedded.

They run on microcontrollers or tiny microprocessors.

The scheduling topic also said that a real-time system lives and dies on scheduling. That's true, because the wrong scheduling choices guarantee failure.

Meeting deadlines in a real-time system is a weakest-link kind of problem.

If any one of the things to be done takes "too long", the deadline is missed.



To get specific about a real-world implementation: **FreeRTOS**.

A free real-time operating system.

We probably need little motivation for why we want this rather than run bare-metal.

The FreeRTOS manual describes it as a “library that provides multi-tasking capabilities to what would otherwise be a single-threaded, bare-metal application”.

This means it does not contain things like device drivers (hardware abstraction) or an I/O framework.

Threads are tasks to do in the system.

They do not follow the task-hierarchy model of parent and child that we see from UNIX(-like) systems.

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode, /* Function to run */  
    const char * const pcName, /* Name for the task */  
    configSTACK_DEPTH_TYPE usStackDepth, /* Size of the stack */  
    void * pvParameters, /* void* argument to be passed to the function */  
    UBaseType_t uxPriority, /* priority */  
    TaskHandle_t * pxCreatedTask /* Task handle, like pthread_t */  
);
```

Compare against `pthread_create()`.

Task states:

- Running
- Ready
- Blocked
- Suspended
- Deleted

FreeRTOS also has an idle task that runs at the lowest possible priority.

It uses the idle task to clean up deleted tasks by invoking the function `prvCheckTasksWaitingTermination()`.

FreeRTOS does not offer a virtual memory scheme; it instead has five heap allocator approaches and they have to be chosen at compile time:

- 1 heap_1: deterministic memory allocation where memory is allocated by the kernel during setup.
- 2 heap_2: best-fit allocation but deallocation does not perform coalescence. It's considered deprecated now.
- 3 heap_3: uses dynamic allocation with `malloc()` and `free()` which it tries to band-aid to be thread-safe by suspending the scheduler while they run.
- 4 heap_4: first-fit with coalescence.
- 5 heap_5: same as heap_4 but across non-contiguous regions.

None of those bring to the party anything like virtual memory, so there are no memory protections.

Trade off overhead vs safety.

Is this an acceptable tradeoff? For a microcontroller scenario, it's easy to say yes.

This also applies to stack overflowing.

The stack size is a design choice in the setup of the system, and if you choose a value that is too small then tasks will overflow their area.

Stack overflows in these scenarios result in corruption of other parts of memory that may be hard to notice until serious damage is done.

A partial solution to this is another form of canary.

Mark the end of the stack with the magic value 0xa5a5a5a5.

FreeRTOS also can calculate whether the saved stack pointer is within the designated stack area for the task.

FreeRTOS uses Fixed Priority Preemptive Scheduling; which can be with or without time slicing.

This is very similar to the Highest Priority, Period algorithm.

The number of priorities is capped at word size, and rather like the $O(1)$ scheduler, this permits finding the first ready task in constant time.

(There is also co-operative scheduling)

Why not EDF when that is provably optimal?

No easy way to get this information into the scheduling routine.

Why not design the system this way?

FreeRTOS favours queues (i.e., message passing) as the mechanism for IPC.

It uses copy semantics: the message is copied into the queue, byte for byte.

There are a few reasons for this related to memory safety.

When multiple threads are waiting, follow scheduler order for unblocking.

In a world of no memory protection, of course, all memory is shared memory.
Making it explicit is much easier to audit.

Mutual exclusion can be done with disabling interrupts.

```
void vPrintString( const char *pcString )
{
    /* Write the string to stdout, using a critical section as a
       crude method of mutual exclusion. */
    taskENTER_CRITICAL();
    {
        printf( "%s", pcString );
        fflush( stdout );
    }
    taskEXIT_CRITICAL();
}
```

But you remember why it is bad, right?

FreeRTOS has mutex, binary semaphore, and general (counting) semaphore options.

The mutex has priority inheritance but the binary semaphore does not.

Unblocking takes place according to task priority.

Microcontrollers and tiny embedded systems also frequently have tiny batteries, unless of course they run when plugged in.

The OS typically has a timer tick, but if there is nothing to do right now, disabling that is actually a way to save power.

Waking up to handle the tick every x milliseconds is a waste of effort and energy.

FreeRTOS offers the idea of tickless idle: just wait in idle mode without waking up for ticks periodically until the next scheduled event.

So if we know that the next task will become ready after x time, what if we just sleep until x time has elapsed and not wake up in between?

Of course, if there is an interrupt, that will wake up the system.

Interrupt handling is a critical part of most embedded RTOS implementations.

How quickly the event is handled is critical to the idea of meeting the deadline.

There's nothing new to say about where interrupts come from or what one looks like; only how they are handled.

Interrupts trigger interrupt service routines (ISRs) which process the event in some way.

ISRs take precedence over normal task execution, and there is always pressure to keep the handler execution time as short as possible.

Use interrupt handling to set up a task to handle the event later. Setting up the task to run later is known as **deferred interrupt processing**.

Remember we talked about the idea that invoking a function in a signal handler requires checking that the function has a designation *async-signal-safe*?

FreeRTOS API is divided into functions that either do or do not contain the words FromISR in their name.

Key design decision.

```
BaseType_t xQueueSendToBack(  
    QueueHandle_t xQueue,  
    const void * pvItemToQueue,  
    TickType_t xTicksToWait /* How long to block when  
                                trying to add if the queue is full */  
);  
  
BaseType_t xQueueSendToBackFromISR(  
    QueueHandle_t xQueue,  
    const void * pvItemToQueue,  
    BaseType_t * pxHigherPriorityTaskWoken /* Set to true if enqueueing  
                                                has unblocked something  
                                                higher priority */  
);
```

Note the difference in the final parameter.

Unusually, software timers in FreeRTOS are tasks rather than interrupts.

At each timer tick, the OS will then figure out whether the timer task should become ready to run and change its state accordingly.

FreeRTOS makes a provision for the difference in time between submitting a command and its processing.

All of the kernel implementation of FreeRTOS is located in just 7 C source files.

That's pretty minimal, and it's remarkably effective for this rather small footprint.
Yes, the memory management strategies are outside of that.

The Linux Kernel added the mode `PREEMPT_RT` in 2024 (after like 20 years of work to get it accepted) that lets Linux be used for hard-real-time tasks.

Why not use that? For a bigger system, sure!

Linux itself is large & complex and expects things like memory management units.

That's too much overhead for a microcontroller scenario.

FreeRTOS fits nicely in a smaller space.

Is this enough for life- and safety-critical systems?

We might perhaps like more assurance.

A variant of FreeRTOS, called SAFERTOS, follows the same functional model but has a different implementation (no open-source code) that is externally verified.

This is what you would want to use if you really intended to build such a system.

It costs money to get a certified OS, but paying for assurance is part of building a reliable system.

Just as meeting deadlines is a weakest-link problem, every component in a life-and safety critical system needs assurance.

Otherwise, we cannot say that the whole system will perform as expected.